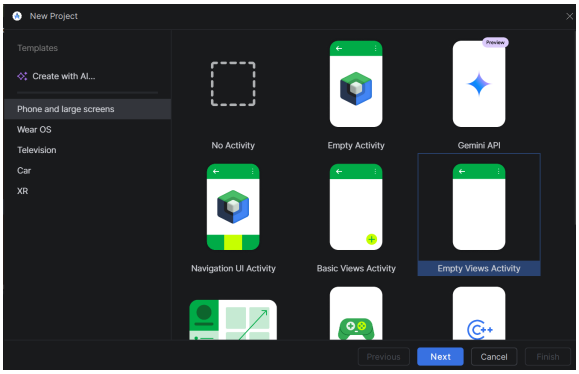
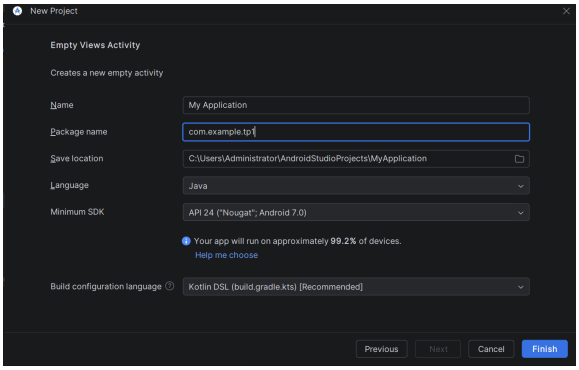
PW 1

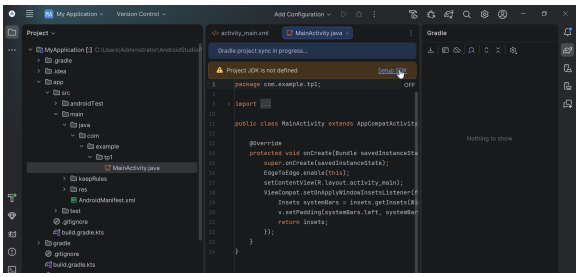
1 Project Creation & Setup

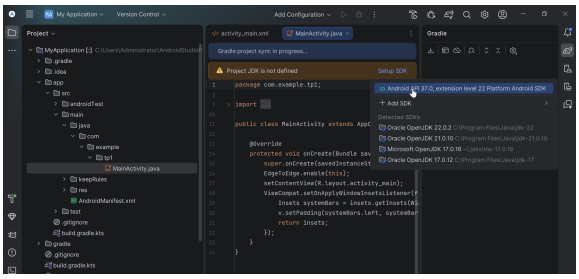
1. Creates new project
2. Choose empty views activity click next
3. Set name of application, package and choose the language java or kotlin and click finish



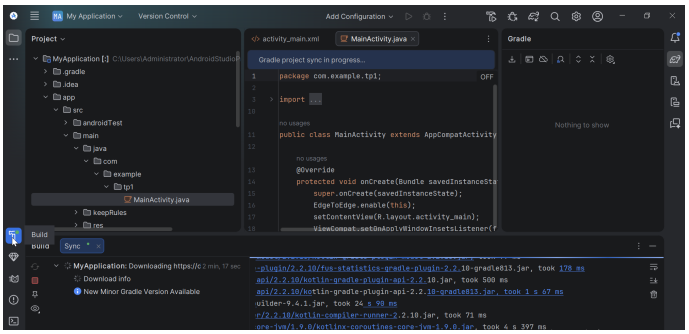


4. Setup the **SDK**





5. gradle will automatically start downloading we can check progression in the build pannel



2 Project Structure

2.1 manifests

The `manifests/` folder contains the `AndroidManifest.xml` file, which houses all activities, intents, and the main entry point of our application. Even if an activity is coded, if it is not declared in the manifest, Android won't be able to access or run it.

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <!-- this set MainActivity as the entry point --!>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

2.2 Kotlin+java

this folder contains all our activities code and tests, we will focus on acitivies code rather than test like the MainActivity :

MainActivity.java

```java
package com.example.tp1;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {


    // override a method which is executed each time this activity's life cycle is onCreate
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState); // calls the parent onCreate() method
        EdgeToEdge.enable(this); // makes the app frame just below system top bar

        setContentView(R.layout.activity_main); // link xml view to the java code

        // lambda function that pushes content inside the frame so they don't overlap
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

2.3 res

`res/` is the resources folder containing:

- `drawable/` all multi-media files (sounds, images, etc.).
- `layout/` has all the XML files for the views of our app.
- `mipmap/` holds the app icons and automatically generates multiple sizes according to the screen DPI.
- `values/` has `colors.xml` and `strings.xml` that can be accessed in code, along with `themes.xml` that we can apply to our views.
- `xml/` holds general XML configuration files (such as searchable configurations, preference files, or network security configs).

3 Adding Logs To All Life Cycle State

We will add logs to each life cycle state method :

MainActivity.java

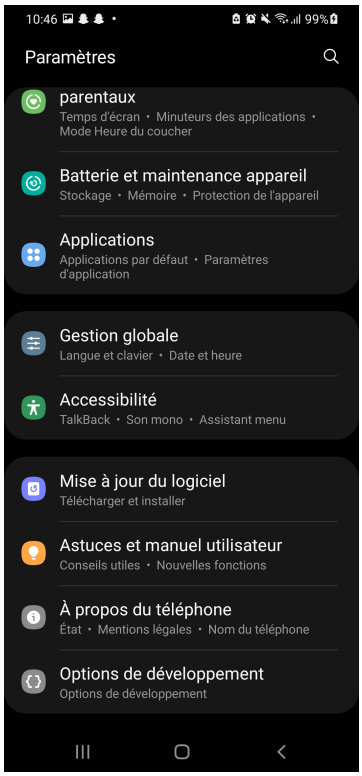
```java
package com.example.tp1;
import static android.content.ContentValues.TAG;
import android.os.Bundle;
import android.util.Log;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

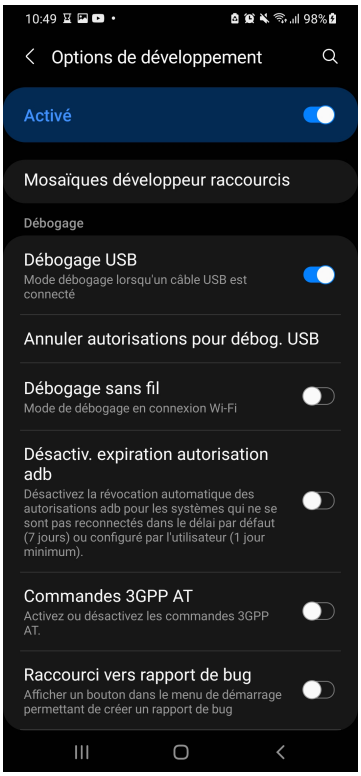
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Log.d(TAG,"onCreate()");
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
    @Override
    protected void onStart() {
        super.onStart();
        Log.d(TAG,"onStart()");
    }
    @Override
    protected void onResume() {
        super.onResume();
        Log.d(TAG,"onResume()");
    }
    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d(TAG,"onRestart()");
    }
    @Override
    protected void onStop() {
        super.onStop();
        Log.d(TAG,"onStop()");
    }
    @Override
    protected void onPause() {
        super.onPause();
        Log.d(TAG,"onPause()");
    }
    @Override
    protected void onDestroy() {
        Log.d(TAG,"onDestroy()"); //log before so it's printed before end of process
        super.onDestroy();
    }
}
```

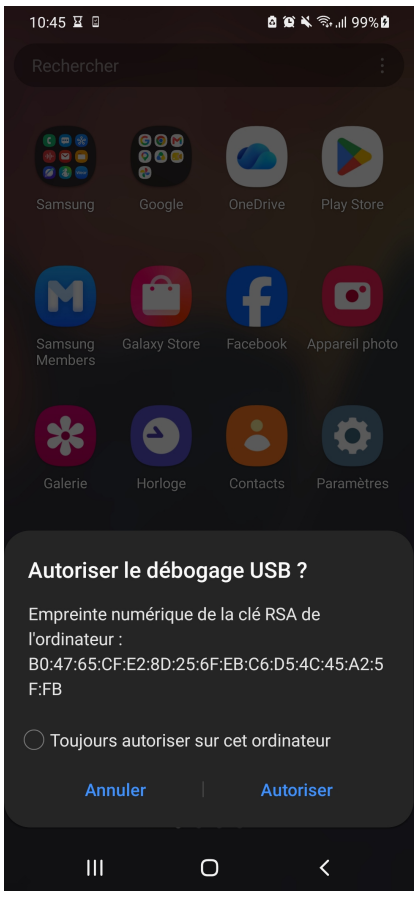
4 Execution & LogCat Output

When trying to execute the code i get an error of not having Windows Hypervisor Platform is not enabled which allows for virtualization of android device in my case since i don't have a lot of storage left i won't be needing this because i'll just use my phone and charger cable to execute app directly, first we have to enable developer mode in the phone and turn on debugging usb :

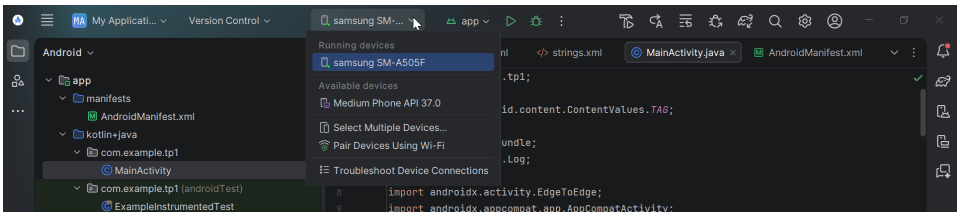




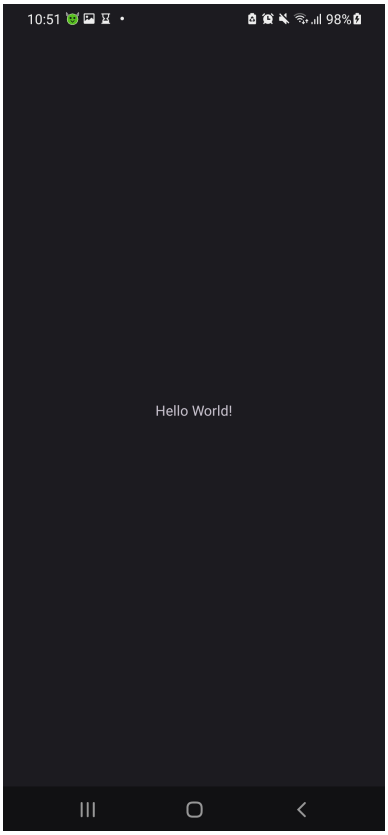
After that we plug the phone with the pc using the charger cable and we authorize usb debugging



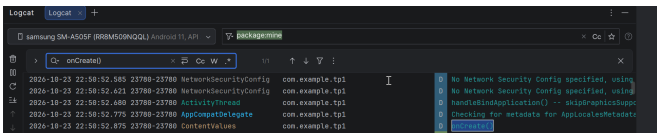
Back to android studio i should be able to have my device as an option

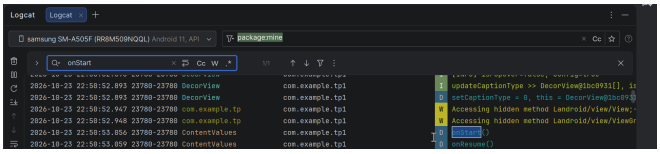


Now when i run the app it will execute directly in my phone

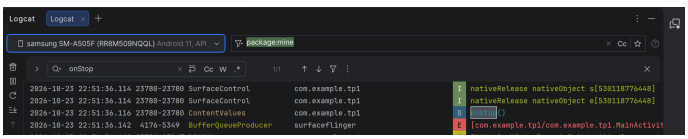


We can check our logs in logcats , when we first launch the app we get state `onCreate() -> onStart() -> onResume()`

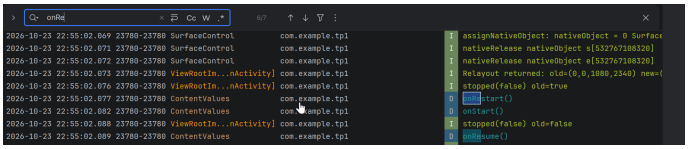




then when my phone goes sleep mod it trigger `onStop()`



when i open my phone it triggers `onRestart() -> onStart() -> onResume()`



when i get notification on top of my app i get `onPause()`



finally when i close the app i get `onDestroy()`

